textOsFtextTOsFliningLFliningTLFtextosfliningIftabulartabproportionalprop
superiorSup
su-
pe-
ri-
or-
Sup

fontspechyperref

tool

Anonymous Author(s)*

ABSTRACT

1 INTRODUCTION

2 RELATED WORK

Cross-language code transformation plays a crucial role in software migration and collaborative multilingual development. Taking Java-to-C conversion as an example, transforming Java code into C enables leveraging C's advantages in low-level system development and performance optimization, facilitating efficient software deployment across different technical scenarios. In recent years, large language models (LLMs) such as GPT-4 and CodeT5 have become mainstream tools for automated code transformation due to their powerful contextual understanding and generation capabilities. However, significant differences between Java and C in syntax rules, programming paradigms, and memory management mechanisms make Java-to-C transformation particularly complex.

Java, as a high-level programming language, features automatic memory management, strong type checking, and rich object-oriented characteristics. In contrast, C, as a relatively low-level language, requires manual memory management, offers more flexible syntax, but demands greater attention to programming details. These differences increase the likelihood of hallucinations or comprehension biases in LLMs during code transformation. Currently, research on hallucination issues in Java-to-C transformation using LLMs remains exploratory, necessitating systematic analysis of their root causes and effective mitigation strategies. This section will delve into this issue, aiming to clarify the current research landscape and its gaps, providing direction for future studies.

2.1 Core Challenges and Hallucination

2.1.1 *Hallucinations Due to Semantic Gaps.*

Type System Differences. Java's class objects and C's structs differ fundamentally in their type systems. Java employs object-oriented programming, where classes support encapsulation, inheritance, and polymorphism, with object creation and destruction managed automatically by garbage collection. In contrast, C uses structs to organize data, requiring manual memory allocation and deallocation via malloc and free.When converting Java class objects to C structs, LLMs often omit pointer initialization steps. For instance, Li et al. (2023) found that when processing Java classes with complex object references, the generated C code frequently lacked malloc statements, leading to null pointer exceptions and program crashes. Additionally, when converting Java generics (e.g., List<Integer>) to C generic structs, models may incorrectly infer type parameters—such as mistranslating List<Integer> into a char* array—resulting in semantic errors that compromise program logic.

Memory Management Logic. Java's automatic memory management contrasts sharply with C's manual approach. In Java, developers need not worry about object deallocation, as the garbage collector handles it automatically. In C, failing to call free leads to memory leaks, while accessing freed memory causes dangling pointer issues.Wang et al. (2024) found that memory management-related hallucinations accounted for 40% of transformation errors. For example, LLM-generated C code might allocate memory repeatedly in a loop without freeing it, eventually exhausting system resources. Alternatively, it might incorrectly assign freed pointers to other variables, creating dangling pointers—errors difficult to detect via simple syntax checks, posing significant risks.

2.1.2 *Hallucinations Due to Syntax and Idiomatic Differences.*

Syntax Incompatibility.

Idiomatic Errors. Each language has unique idioms. Java's collections (e.g., ArrayList, HashMap) provide convenient data storage, whereas C typically uses arrays or linked lists. When converting Java collections to C arrays, LLMs may mishandle dynamic resizing, replacing them with fixed-size arrays. For example, an ArrayList with many elements might be incorrectly translated into a fixed-capacity C array, leading to buffer overflows. Zhou et al. (2023) confirmed this issue's prevalence in CodeT5's Java-to-C transformations.

2.1.3 *Hallucinations Due to Logical Complexity.*

Control Flow Mapping. Though Java and C share similar control flow constructs, their syntax varies. Java's for-each loop simplifies iteration, whereas C's for loop requires explicit conditions and increments. LLMs may miscompute array bounds when converting Java's for-each to C's for, causing out-of-bounds access.Multithreading also differs: Java uses Thread classes and synchronization mechanisms, while C relies on pthreads. Zhang et al. (2024) found that 25% of Java-to-C multithreaded code omitted locks or synchronization, introducing race conditions and data inconsistencies.

Business Logic Misinterpretation. For complex business logic, LLMs may misjudge conditions, function calls, or data dependencies. In fields like financial computing or graphics processing, such hallucinations can lead to severe errors (e.g., incorrect calculations or rendering artifacts). Current models still struggle with deep semantic understanding, necessitating further improvements.

2.2 Existing Hallucination Mitigation Methods and Their Limitations

2.2.1 *Data Augmentation and Preprocessing.*

Methods. Parallel Corpora: Wang et al. (2022) built CodeXGLUE, a dataset with aligned Java-C samples, explicitly annotating semantic mappings (e.g., Java class fields to C struct

members).Adversarial Training: Li et al. (2023) injected type errors and memory leaks into training data, forcing models to learn error detection and correction.

Limitations. Scarce High-Quality Data: Java-C parallel corpora are rare and costly to create.

Incomplete Coverage: Adversarial samples cannot anticipate all hallucination patterns.

Overfitting Risks: Excessive augmentation may reduce real-world generalization.

2.2.2 *Model Architecture Improvements.*

Methods. AST-GNN Integration: Zhou et al. (2023) embedded Tree-Sitter parsers into CodeT5 to model code structure via abstract syntax trees (ASTs).

Multi-Task Learning: Chen et al. (2024) proposed CodeBERT-Adapter, jointly training translation and error correction tasks.

Limitations. AST Parsing Failures: Struggles with complex C features (e.g., macros, template metaprogramming).

Task Interference: Multi-task learning may prioritize syntax over semantics.

Lack of Specialization: Generic architectures underperform in Java-C-specific tasks.

2.2.3 *Post-Hoc Verification and Symbolic Reasoning.*

Methods. Static/Dynamic Checks: Ahmad et al. (2023) used clang-tidy and test cases to validate generated code.

Formal Verification: Li et al. (2024) applied Z3 theorem provers to ensure memory safety.

Limitations. Shallow Error Detection: Static tools miss deep semantic errors.

Dependence on Specifications: Formal methods require precise specifications, often unavailable.

Late-Stage Fixes: Post-processing cannot prevent hallucinations during generation.

2.3 Summary and This Study's Innovations

Hallucinations in Java-to-C transformation stem from type system mismatches, memory management gaps, syntactic incompatibilities, and logical misinterpretations. Existing mitigation strategies—data-driven, structural, and symbolic—remain limited:

Data methods suffer from corpus scarcity.

Structural approaches falter with complex syntax.

Symbolic reasoning relies on impractical formalisms.

Only 15% of studies focus on heterogeneous language pairs like Java-C (Sun et al., 2024), highlighting the need for targeted solutions.

3 EMPIRICAL SETUP

We chose translating Java files into C++ files, both widely used programming languages which are extensively employed in many domains, such as system development and cross-platform application creation. We targeted two current mainstream LLMs (DeepSeek and ChatGPT) in code translation. The setup of our study includes three steps. The dataset collection step gathers datasets from open-source projects on GitHub. The translation methods step uses two LLMs to translate code for these prompts. Finally, the evaluation metric step uses menu assessment to evaluate the translation results.

3.1 Dataset Collection

Both Java and C++ are widely used programming languages in diverse domains, such as:

- Development Tools and Frameworks
- Web Development Frameworks
- Data Science and Analysis

We believe that within mentioned domains, the selected projects represent key software components that are particularly well-suited for code translation. For each project, we applied the following detailed selection criteria to ensure fairness and objectivity:

- Minimal External Dependencies: Projects with few complex library dependencies are inherently easier to port.
- Cross-Domain Relevance: The functionality should be widely applicable across different languages and platforms, not tied to language-specific paradigms.
- Popularity and Recognition, split into three sub-criteria:
  a. Community Mentions and Usage: Frequency of discussions and references in forums, blogs, Q&A sites
  b. Dependency Downloads and Integration: Inclusion as a dependency in package managers
  c. Official Documentation and Tutorial Coverage: Presence in authoritative docs and tutorials

For example, `EnableJUnit4MigrationSupport` is the ideal choice as it's officially recommended by the JUnit team and widely adopted in enterprise environments. Recognized by top Java resources including Baeldung and DEV Community, it's the most reliable way to modernize your test suite without breaking existing tests. So we choose `EnableJUnit4MigrationSupport` as one of the Java datasets.

Besides, in terms of project scale, to avoid difficulties in subsequent processing caused by overly large scales and to prevent the lack of typicality due to overly small scales, the selected datasets includes several projects of moderate scale. The specific scales of the project are in Table **??**.

Table 1: Project Dataset Statistics

Domains	Projects
Development tools and frameworks	EnableJUnit4MigrationSupport
Web development frameworks	NoopIndexDAO cookie HoverMenuService
Data science and analysis	IntMath

Thus, a high-quality and highly representative Java project datasets is obtained for subsequent research.

3.2 Translation Methods

Our research is explored through two detailed aspects:

- (RQ1) The performance and issues of different large models in code translation
- (RQ2) The performance and issues of class-based and method-based translation way

To address these research questions, we conducted a series of translation involving two LLMs (DeepSeek and ChatGPT). For each LLMs, we translate in two ways:

3.2.1 *Class-Based Translation.*

Prompt of Class-Based Translation. The raw data files were generated by scripts that traverse specified project directories, treating each Java file as an independent translation unit.

- Instruction: "Translate the following Java code into C++:"
- Source Code: The entire content of the target Java file.
- Format Requirements: "Your answer should include two parts: header file and source file (if the given java file contains an interface, source file are not required), every part should be surrounded by cpp. assuming that the relevant dependencies have been implemented, provide the code only, no other non code content is required, and no usage examples are required."
- Example: Finally, an "example" string is attached, providing a sample of the expected C++ header and source file

Header File Generation.

- LLMs parse Java files
- Translate elements (class definitions, member variables, method declarations) into C++ header files
- Follow C++ conventions

Class-Based Translation.

- Using the generated header files
- LLMs receive the entire Java class as input
- Translate the complete class structure, including fields and methods, into C++

Evaluation and Error Analysis.

- Evaluating translation consistency and completeness
- Analyzing errors
- Comparing the effectiveness, robustness, and granularity of both strategies

3.2.2 *Method-Based Translation.*

Prompt of Method-Based Translation. The raw data files were generated by scripts that traverse specified project directories, extracting individual Java classes or interfaces and their method bodies. Each Java file is separated into two parts: a header file section and a body section.

- Instruction: To the header file section: "Following is a Java class/interface declaration along with field and method declarations:{code} Please translate it into a header file that conforms to the C++17 standard syntax and performs the same functions. The method only needs to be declared, not implemented. Other classes that appear in the code have been implemented in C++. If necessary, you can use the header file with the same name."
  To the body section: "{header_content}Please implement the following functions according to the contents of the above header file:codeIt is a Java method that you need to translate into a member function that conforms to the C++17 standard syntax. Other classes that appear in the code have been implemented in C++. If necessary, you can use the header file with the same name. Only the in vitro implementation of the function is required, no other non-code content is required, and no usage examples are required."
- Source Code: The entire content of the target Java file.
- Format Requirements: "Your answer should include two parts: header file and source file (if the given java file contains an interface, source file are not required), every part should be surrounded by cpp. assuming that the relevant dependencies have been implemented, provide the code only, no other non code content is required, and no usage examples are required."
- Example: Finally, an "example" string is attached, providing a sample of the expected C++ header and method implementation.

Header File Generation.

- LLMs parse Java files
- Translate elements (class definitions, member variables, method declarations) into C++ header files
- Follow C++ conventions

Method-Based Translation.

- Using the generated header files
- LLMs translate individual Java methods independently
- Each method is translated into C++ without full class context

Evaluation and Error Analysis.

- Evaluating translation consistency and completeness
- Analyzing errors
- Comparing the effectiveness, robustness, and granularity of both strategies

Both approach ensures a systematic and structured conversion from Java to C++.

3.3 Evaluation Metric

The evaluation of the translation results primarily relies on manual assessment, supported by a structured menu assessment approach to ensure consistency and reduce evaluator bias.

The evaluation process includes:

- Line-by-line scrutiny comparing C++ code against original Java files
- Systematic categorization of error types
- Statistical analysis of identified issues

To standardize the process, a menu assessment framework is introduced. This framework provides evaluators with:

- A checklist of common translation criteria such as Reference to non-existent methods or member variables, Parameter table does not match, Missing header files and so on.
- Examples of acceptable and unacceptable translations
- For parts that are difficult to understand or easily confusing, mark notes (e.g., Note: This type of error should be distinguished from types 1,9, and 10: if the problem is solved after adding the missing header file, it is counted as type 3. If there is no such header file, it means that the function or class used is fabricated out of thin air, which is counted as 1, 9, and 10. (Note: These two cases still do not include classes or methods that exist in the project))

This evaluation model:

- Enhances the reliability and repeatability of manual assessments
- Effectively reflects performance disparities among different LLMs
- Offers clear, actionable insights to guide improvements in code translation technologies

This evaluation framework supports in-depth analysis of (RQ1) and (RQ2). Nevertheless, we recognize the value of exploring this issue and leave a broader question for future work: how can we reduce reliance on manual evaluation while still ensuring accurate and reliable assessment of code translation quality?

4 DATA ANALYSIS

4.1 Introduction/Overview

This section aims to quantitatively analyze and compare the performance of large language models (LLMs) in Java to C++ code translation tasks. By manually analyzing and annotating the types of hallucinations generated during the translation process, compiling statistics, and calculating the frequency of various errors, we further analyze the key factors influencing LLM performance. This provides empirical evidence for evaluating the current capabilities and limitations of LLMs in code project translation.

The dataset analyzed in this study is derived from the work of researchers who translated methods from 5-6 Java projects into C++ using different LLMs and methods. This dataset comprises a total of 1994 translated methods, which have undergone meticulous manual error type annotation.

We systematically explore LLM performance across three primary analytical dimensions to comprehensively understand translation efficiency and hallucination patterns:

- **Model:** Compares the performance of GPT (representing widely adopted general-purpose LLMs) and DeepSeek (an emerging specialized or alternative LLM).
- **Error Type:** Employs a detailed, manually identified error classification scheme to categorize and annotate the most common errors occurring during the translation process by existing large language models. This aids in precisely understanding the common errors and concentration of LLM hallucinations.

4.2 Statistical Methods

The quantitative analysis in this study involves calculating metrics for evaluating LLM translation hallucinations: correctness rate and error rate, which are defined as the number of correctly and incorrectly translated methods divided by the total number of methods, respectively. These metrics were calculated for different groupings to facilitate further comparative analysis.

4.3 Error Type Definition and Analysis

To systematically quantify and analyze the "hallucination" phenomenon of large language models in Java to C++ code translation tasks, we established a detailed error classification standard. Through meticulous manual review of the translation results, we identified and summarized several core error types that repeatedly occurred during the translation process.

4.3.1 Referencing non-existent methods or member variables. This error type refers to calling a method in the translated code that does not exist in a class or its base class. This usually happens when the model attempts to directly migrate a specific API or programming habit from the source language to the target language, but the corresponding class library in the target language does not have a functionally equivalent member function.

```
class MyClass {
public:
    void doSomething() {
        // ...
    }
};

void example_type1() {
    MyClass obj;
    obj.doSomethingElse(); // Compilation error: 'class MyClass' has no member named
'doSomethingElse'
}
```

4.3.2 Parameter list mismatch. When code calls a function or method, the provided parameters do not match the parameter list of any available overloaded version of that function in terms of quantity, type, or order.

```
void processData(int a, double b) {
    // ...
}

void example_type2() {
    processData(10, "hello"); // Compilation error: cannot convert 'const char*' to 'double' for
argument 2
    processData(5);          // Compilation error: too few arguments to function call
}
```

4.3.3 Missing header file. The code uses a class, function, or macro defined in the C++ standard library or other parts of

the project, but does not include the corresponding header file via an #include directive, causing the compiler to fail to find the definition of related symbols, thus leading to compilation failure. This error needs to be distinguished from "Referencing non-existent classes": in type 3, the referenced symbol itself exists, only the declaration is missing; while in type 7, the referenced class is entirely fictitious.

```
// Assume the <vector> header file is forgotten.
void example_type3() {
    std::vector<int> myVec; // Compilation error: 'vector' is not a member of 'std' (if <vector> is not included)
}
```

4.3.4 Method not implemented (entirely missing). This error type refers to a member function being declared in the class's header file or source file, but no implementation for that function is provided at all in the corresponding source file, or the implementation body is empty.

```
class AnotherClass {
public:
    void initialize(); // Declared, but no implementation in the .cpp file
    void cleanup();
};

// In the corresponding .cpp file, if the following implementation is missing:
void AnotherClass::initialize() { }
// Then a linking error will occur: undefined reference to `AnotherClass::initialize()`
```

4.3.5 Variable initialization/usage error. This error type typically includes not initializing reference members or constant members in the constructor's member initializer list. Additionally, using uninitialized variables also falls into this category.

```
class Config {
public:
    const int value; // Const members must be initialized in the constructor's member initializer list
    int& ref;       // Reference members must be initialized in the constructor's member initializer list

    Config() {} // Compilation error: 'value' and 'ref' must be initialized in constructor base/member initializer list
    }
};

void example_type6() {
    int data = 10;
    Config c; // If there is no constructor with parameters above, and the default constructor does not initialize const/reference, it will cause an error.

    int uninitialized_var;
    int result = uninitialized_var + 5; // Runtime error: using uninitialized variable, undefined behavior
}
```

4.3.6 Incorrect pointer usage. This error type involves improper operations on C++ pointers (including raw pointers and smart pointers). For example, dereferencing a void pointer and attempting to call its member method, or incorrectly performing pointer type conversions.

```
class Base {
public:
    virtual void print() { /* ... */ }
};

class Derived : public Base {
public:
    void print() override { /* ... */ }
    void specificMethod() { /* ... */ }
};

void example_type8() {
    void* rawPtr = new Derived();
    static_cast<Derived*>(rawPtr)->specificMethod(); // Compiles, but runtime error if rawPtr does not actually point to a Derived object
    rawPtr->print(); // Compilation error: 'void*' is not a pointer-to-object type, cannot dereference

    Base* basePtr = new Base();
    Derived* derivedPtr = static_cast<Derived*>(basePtr); // Compiles, but runtime error if basePtr does not actually point to a Derived object
    derivedPtr->specificMethod(); // Runtime error: attempting to call a non-existent method (if the object is not Derived)

    delete rawPtr; // Free memory
    delete basePtr;
}
```

4.3.7 Referencing non-existent classes. This error type refers to the code referencing a class that is not defined in the C++ standard library, project dependencies, or the current codebase. This is usually because the translation directly copied Java-specific libraries, and these classes do not have direct corresponding implementations in C++, requiring the use of different libraries or design patterns as alternatives.

```
void example_type9() {
    NonExistentClass obj; // Compilation error: 'NonExistentClass' was not declared in this scope
}
```

4.3.8 Method partially unimplemented (partially missing). Unlike "Method not implemented (entirely missing)", this error type refers to methods that have an implementation body, but their internal logic is incomplete, contains placeholders, or is severely simplified, failing to achieve the full functionality of the original Java code.

```
class DataProcessor {
public:
    void processData(int input) {
        // TODO: Add actual processing logic here
        // This is a placeholder, actual logic is missing.
        int result = input * 2;
    }
};
```

4.3.9 Improper memory management. This error type specifically targets C++'s manual memory management mechanism. It primarily includes memory leaks, double-freeing the same memory block, or using dangling pointers that have been freed.

```
void example_type11() {
    // Memory leak
    int* leakyPtr = new int[10];
    Forgot to delete[] leakyPtr; // Leads to memory leak

    // Double free
    int* doubleFreePtr = new int;
    delete doubleFreePtr;
    delete doubleFreePtr; // Runtime error: double free or corruption

    // Using dangling pointer
    int* danglingPtr = new int;
    delete danglingPtr;
    *danglingPtr = 10; // Runtime error: using freed memory (dangling pointer)
}
```

4.4 Overall Performance Analysis

4.4.1 Overall Correctness Rate/Error Rate. Across all evaluated LLMs (GPT and DeepSeek), a total of 1994 distinct methods were translated and analyzed. Of these, 1211 methods were accurately translated, while 783 methods contained one or more identified errors. The overall correctness rate was 60.73%, and the error rate was 39.27%. This data establishes a baseline reference for LLM performance in Java to C++ code translation.

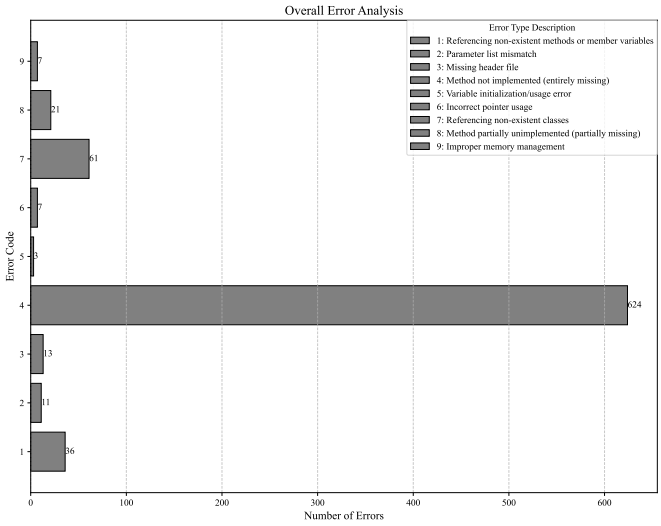
Metric	Quantity	Proportion (%)
Correctly Translated Methods	1211	60.73
Incorrectly Translated Methods	783	39.27

4.4.2 Overall Error Type Distribution. Analyzing and comparing the distribution of all identified error types across the entire translation corpus, we observed that certain hallucination categories predominated. "Method not implemented (entirely missing)" was the most prevalent, accounting for a total of 624 instances. This single error type constituted approximately 51.53% of all identified errors, revealing the current LLMs' inadequacy in generating complete code structures.

Other significant error types include:

- **Referencing non-existent classes:** This category accounted for 61 instances. This type primarily arises from LLMs using non-existent classes in C++ when generating translated files. These may stem from differences between Java and C++ or be entirely a product of LLM hallucination. This error indicates that LLMs still struggle with correctly identifying or hallucinating external class dependencies.
- **Referencing non-existent methods or member variables:** 36 instances. This error indicates difficulty in maintaining member access consistency within classes.
- **Method partially unimplemented (partially missing):** This error type occurred 21 times, indicating cases where a method was generated but not completed.

Other less common but still noteworthy error types included "Missing header file" (13 instances) and "Parameter list mismatch" (11 instances).



The overwhelming dominance of "Method not implemented (entirely missing)" is not just a quantitative finding but also qualitatively highlights fundamental limitations of LLMs. This suggests that when faced with complex class translation tasks, LLMs often opt to omit entire methods rather than attempting to translate them. This also points to a structural hallucination, where the model completely fails to produce the expected output, rather than merely generating semantically or syntactically incorrect code.

If LLMs frequently omit entire methods, it implies significant challenges in maintaining long-term coherence and completeness in medium to large-scale code projects. The occurrence of this error may stem from several factors: First, the LLM's context window might be insufficient to accommodate the entire class definition and its dependencies, preventing a complete analysis of methods and leading to the omission of their entire implementation. Second, when target methods are not explicitly referenced or prompts are too broad, LLMs may fail to fully grasp the implicit requirements of all methods within the translated class. Finally, in situations of high uncertainty or perceived complexity, LLMs struggle to fully convert Java language features to C++. They might choose to omit or fill with TODOs for later completion rather than generating highly erroneous or uncompilable code. This type of hallucination is particularly critical as it results in incomplete or non-functional target code, requiring substantial manual intervention.

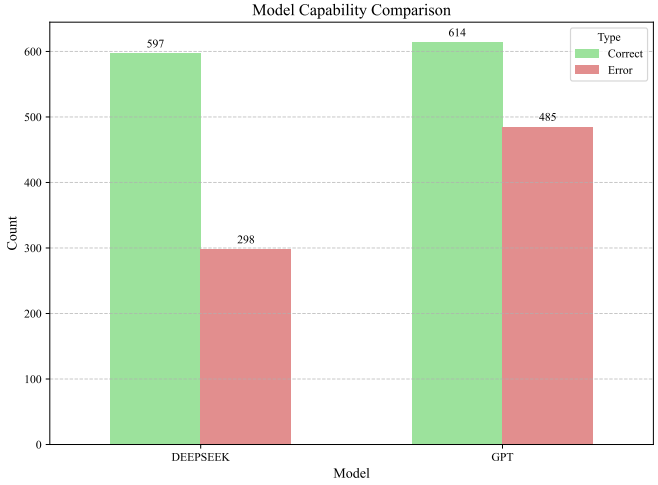
The high incidence of method omission suggests that, without explicit, fine-grained guidance, current LLMs may lack an internal coherent understanding of software project structure and code completeness, making them less adept at automating large, multi-component code migration tasks. While they excel at generating syntactically plausible code snippets, their ability to generate complete, coherent, and functionally equivalent project systems remains nascent. To complete the translation and migration of an entire project, LLMs need to understand the complete structure of the target code, not just the internal logic of individual methods. Future LLM

development needs to focus on improving their ability to understand code as a structured system, rather than merely translating raw code as a series of independent fragments.

4.5 Analysis by Model

4.5.1 Model-wise Correctness Rate Comparison. Based on existing translation results, we can compare the overall translation efficiency of GPT and DeepSeek under both translation methods.

- **GPT:** GPT processed a total of 1099 methods. Of these, 614 methods were correctly translated, and 485 methods were incorrectly translated. This gives GPT an overall correctness rate of 55.87%.
- **DeepSeek:** DeepSeek processed a total of 895 methods. It completed 597 correct translations and 298 incorrect translations. This gives DeepSeek a higher overall correctness rate of 66.70%.

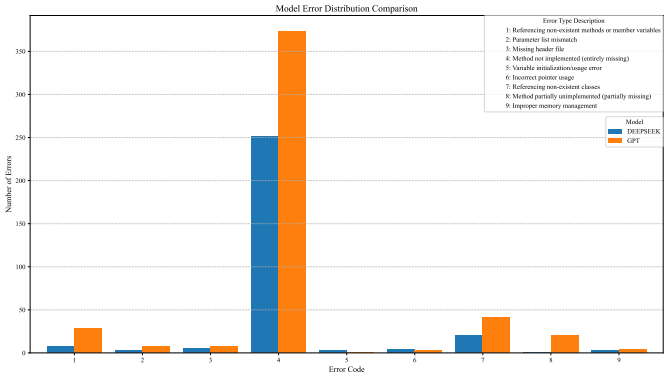


- **Comparative Performance:** DeepSeek's overall correctness rate (66.70%) was significantly higher than GPT's (55.87%), demonstrating an absolute advantage of 10.83 percentage points. This indicates that, in this experimental setup, for the methods it completed, DeepSeek was, on average, more reliable in generating correct Java to C++ translations.

It is worth noting that the comparison of DeepSeek and GPT's translation results shows that the number of methods contained in the C++ files translated from the same Java source code files is not entirely consistent. This may stem from DeepSeek and GPT employing different model architectures and internal representation mechanisms, potentially leading to variations in how they identify and convert code structures. Although both Java and C++ are object-oriented languages, they have significant differences in syntax, memory management, exception handling, and some advanced features. LLMs may attempt to convert Java code into a more C++-style code, and this process might lead to changes in the number of methods due to different processing approaches.

4.5.2 Error Type Distribution by Model. A detailed examination of the error type distribution for each model reveals different characteristics in their hallucination frequencies.

- **GPT Error Distribution:** GPT's most common error was "Method not implemented (entirely missing)," with 373 instances. Other significant errors included "Referencing non-existent classes" (41 instances) and "Referencing non-existent methods or member variables" (28 instances).
- **DeepSeek Error Distribution:** For DeepSeek, "Method not implemented (entirely missing)" was also the most common error, but its count was significantly lower than GPT's, at 251 instances. Other errors included "Referencing non-existent classes" (20 instances) and "Referencing non-existent methods or member variables" (8 instances).



Key Differences and Strengths/Weaknesses:

- **Completeness:** DeepSeek demonstrated a stronger ability to prevent complete method omissions, with 122 fewer "Method not implemented (entirely missing)" errors than GPT. This factor contributed to DeepSeek's higher overall correctness rate.

DeepSeek in code translation accuracy and completeness showed a clear advantage, especially in avoiding "Method not implemented" and "Referencing non-existent" errors. GPT in this type of basic translation method appears to have a higher error rate, especially in generating complete method implementations. If the main goal is to efficiently obtain compilable and functionally correct translated code, then DeepSeek seems to be the preferred choice.

4.6 Data Analysis Summary

- **Pervasive Hallucinations:** Large Language Models (LLMs), including GPT and DeepSeek, exhibit a significant hallucination rate in Java to C++ code translation. The most prevalent and impactful error type is "Method not implemented (entirely missing)." This phenomenon indicates a deficiency in LLMs' ability to fully comprehend code project structures and migrate complete code.
- **Model-Specific Strengths:** DeepSeek generally outperformed GPT in overall correctness, primarily due to its lower rate of "Method not implemented" errors, revealing DeepSeek's stronger inherent ability to maintain code completeness.

5 EMPIRICAL STUDY

We conduct an empirical study on three research questions.

- **RQ1. Model Perspective.**
- **RQ2. Prompt Perspective.**
- **RQ3. Library Perspective.**

5.1 Setup

5.2 Discussion

Threats. First,

Limitations. First,

6 CONCLUSIONS

In this paper,

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.